

# CampusGPT

AI-Powered Academic Operating System using Small Language Models & RAG

| Field         | Details                                       |
|---------------|-----------------------------------------------|
| Project Title | CampusGPT – Localized Academic AI Assistant   |
| Technology    | Spring Boot · React · PostgreSQL · Ollama LLM |
| Domain        | Unsupervised Machine Learning · NLP · RAG     |
| Type          | Full-Stack Web Application                    |
| Report Date   | April 2026                                    |

## 1. Project Overview

CampusGPT is a **privacy-first, fully offline AI study assistant** built for university students. It runs entirely on the user's local machine — no cloud API, no data leakage. Students upload their course PDFs, and the system answers questions, generates exam answers, creates revision notes, and provides viva preparation — all powered by a locally running Large Language Model (LLM).

The core innovation is a production-grade **Retrieval-Augmented Generation (RAG)** pipeline combined with a full **Unsupervised Learning analytics engine** that visualizes how the AI "understands" a student's documents using K-Means clustering, PCA, and anomaly detection.

## 2. Technology Stack

| Layer    | Technology                                              | Purpose                                    |
|----------|---------------------------------------------------------|--------------------------------------------|
| Frontend | React 18, TypeScript, Vite, Tailwind CSS, Framer Motion | Interactive UI with real-time streaming    |
| Backend  | Spring Boot 3.2, Java 17, Spring Security, WebFlux      | REST API, JWT auth, RAG pipeline           |
| Database | PostgreSQL 16 + pgvector extension                      | Stores documents, embeddings, chat history |
| AI / LLM | Ollama · llama3.2:3b · nomic-embed-text                 | Local inference & vector embeddings        |
| DevOps   | Docker, Docker Compose, Nginx                           | One-command deployment on any machine      |

## 3. System Architecture

### 3.1 RAG Pipeline (ChatService.java)

The RAG pipeline is the heart of the system. When a student asks a question:

- **Step 1 – Embed:** The question is converted to a 768-dimensional vector using `nomic-embed-text` via Ollama.
- **Step 2 – Hybrid Search:** PostgreSQL runs a combined **semantic search** (pgvector cosine similarity) and **keyword search** (tsvector full-text) using Reciprocal Rank Fusion (RRF) to score results.
- **Step 3 – MMR Re-ranking:** Maximal Marginal Relevance filters results for diversity to avoid redundant context.
- **Step 4 – Prompt Construction:** A mode-specific system prompt is assembled with the retrieved chunks as context.
- **Step 5 – SSE Streaming:** The prompt is sent to `llama3.2:3b` and each generated token is streamed back to the browser in real-time using Server-Sent Events.

3.2 Unsupervised Learning Engine (EmbeddingAnalyticsService.java)

Implemented entirely in pure Java with zero external ML libraries — all algorithms written from scratch:

| Algorithm             | Implementation                                              | Purpose                                          |
|-----------------------|-------------------------------------------------------------|--------------------------------------------------|
| K-Means Clustering    | K-Means++ initialization, 50 iterations, Euclidean distance | Groups document chunks into thematic clusters    |
| PCA (2D Projection)   | Power iteration for top-2 eigenvectors, mean-centred data   | Reduces 768-D embeddings to 2D for visualization |
| t-SNE Simulation      | Category-aware perturbation of PCA projections              | Non-linear topology-preserving visualization     |
| LOF Anomaly Detection | k-NN local reachability density, normalized score           | Flags irrelevant/noise chunks in uploaded PDFs   |
| Silhouette Score      | Mean intra vs inter cluster distance ratio                  | Validates cluster quality (Unit VI metric)       |
| WCSS / Inertia        | Within-cluster sum of squares                               | Elbow method support for optimal k selection     |

4. Key Features

4.1 Smart Chat Modes

Six expert AI personas, each with a distinct system prompt:

- **Explain Concept** — Feynman Technique explanation with analogies
- **10-Mark Answer** — University exam structured answer
- **Short Notes** — High-density bullet-point revision notes
- **Viva Prep** — Oral exam Q&A with model answers
- **Revision Blast** — Ultra-compact 10-minute pre-exam summary
- **Exam Strategy** — Topic prioritization and study plan

4.2 Document Intelligence

- PDF upload with Apache PDFBox text extraction
- Semantic chunking and embedding via nomic-embed-text (768 dimensions)
- Stored as pgvector-compatible TEXT format in PostgreSQL

- Per-user document scoping — each student only sees their own data

### 4.3 Analytics Dashboard

- Interactive 2D scatter plot of document embeddings (PCA and t-SNE views)
- Colour-coded cluster assignments with auto-generated cluster labels
- Anomaly highlighting (LOF score > 0.65 flagged in red)
- Metrics panel: Silhouette Score, Inertia, Variance Explained (PC1 + PC2)

### 4.4 Security (OWASP Compliant)

- BCrypt password hashing
- Stateless JWT authentication (RS256)
- Input sanitization to prevent prompt injection
- Bucket4j rate limiting (10 req/min on auth endpoints, 60 req/min authenticated)
- All data stays on local hardware — zero cloud dependency

## 5. Unsupervised Learning Mapping

| Unit    | Topic                      | Implementation in CampusGPT               |
|---------|----------------------------|-------------------------------------------|
| Unit II | Partition-Based Clustering | K-Means++ on 768-D embedding vectors      |
| Unit IV | Dimensionality Reduction   | PCA via power iteration; t-SNE simulation |
| Unit V  | Anomaly Detection          | LOF-style local density scoring           |
| Unit VI | Cluster Validation         | Silhouette Score + WCSS/Inertia metrics   |

## 6. How to Run the Project (For Evaluator)



**No Java, Node.js, or PostgreSQL installation needed.**  
Everything runs inside Docker containers. You only need **Docker Desktop**.

### Step 1 — Install Docker Desktop

Download and install from: <https://www.docker.com/products/docker-desktop>

It is free and available for Windows, Mac, and Linux. After installing, make sure Docker Desktop is **running** (look for the whale icon in the system tray/menu bar).

### Step 2 — Open Terminal in Project Folder

- **Mac:** Right-click the `SLM` folder → "New Terminal at Folder"
- **Windows:** Open the `SLM` folder in File Explorer → click the address bar → type `cmd` → press Enter

### Step 3 — Run One Command

```
docker compose up --build
```

Docker will automatically build and start 4 services:

- **postgres** — Database with pgvector
- **ollama** — Local AI model server (downloads models automatically)
- **backend** — Spring Boot API on port 8080
- **frontend** — React app served via Nginx on port 3000

 **First run takes 5–10 minutes** — Ollama needs to download the AI models (~2 GB). Internet required for first run only. Subsequent runs start in under 30 seconds.

Step 4 — Open the App

Once you see `campusgpt-frontend` in the logs, open your browser and visit:

```
http://localhost:3000
```

Step 5 — Test the App

- 1 Click **Sign Up** and create a new account.
- 2 Go to **Upload** and upload any PDF (lecture notes, textbook chapter, etc.).
- 3 Go to **Chat**, select a Smart Mode (e.g. "Explain Concept"), and ask a question about the PDF.
- 4 Go to **Analytics** to see the K-Means clusters and PCA visualization of your document.

To Stop the Application

```
docker compose down
```

7. Project File Structure

| Path                                                                                      | Description                                    |
|-------------------------------------------------------------------------------------------|------------------------------------------------|
| <code>docker-compose.yml</code>                                                           | Orchestrates all services — single entry point |
| <code>backend/src/main/java/com/campusgpt/chat/ChatService.java</code>                    | Full RAG pipeline with MMR re-ranking          |
| <code>backend/src/main/java/com/campusgpt/analytics/EmbeddingAnalyticsService.java</code> | All unsupervised learning algorithms           |
| <code>backend/src/main/java/com/campusgpt/document/DocumentService.java</code>            | PDF parsing, chunking, embedding               |
| <code>backend/src/main/resources/application.yml</code>                                   | All configuration via environment              |

|                                      |                                    |
|--------------------------------------|------------------------------------|
|                                      | variables                          |
| frontend/src/pages/ChatPage.tsx      | Real-time SSE<br>streaming chat UI |
| frontend/src/pages/AnalyticsPage.tsx | Interactive analytics<br>dashboard |